

Herencia e Interfaces

Ejercicio 1. (Herencia)

Queremos gestionar la venta de entradas (no numeradas) de **Expocoches** que tiene 3 zonas, la sala principal con 1000 entradas disponibles, la zona de compra-venta con 200 entradas disponibles y la zona vip con 25 entradas disponibles. Hay que controlar que existen entradas antes de venderlas.

La clase Zona con sus atributos y métodos se muestra a continuación:

```
public class Zona {
    private int entradasPorVender;
    public Zona(int n){
        entradasPorVender = n;
    }
    public int getEntradasPorVender() {
        return entradasPorVender;
    }
    /**
     * Vende un número de entradas.
     * <p>
     * Comprueba si quedan entradas libres antes de realizar la venta.
     *
     * @param n número de entradas a vender
     */
    public void vender(int n) {
        if (this.entradasPorVender == 0) {
            System.out.println("Lo siento, las entradas para esa zona están agotadas.");
        } else if (this.entradasPorVender < n) {
            System.out.println("Sólo me quedan " + this.entradasPorVender + " entradas para esa zona.");
        }
        if (this.entradasPorVender >= n) {
            entradasPorVender -= n;
            System.out.println("Aquí tiene sus " + n + " entradas, gracias.");
        }
    }
}
```

El menú del programa debe ser el que se muestra a continuación. Cuando elegimos la opción 2, se nos debe preguntar para qué zona queremos las entradas y cuántas queremos. Lógicamente, el programa debe controlar que no se puedan vender más entradas de la cuenta.

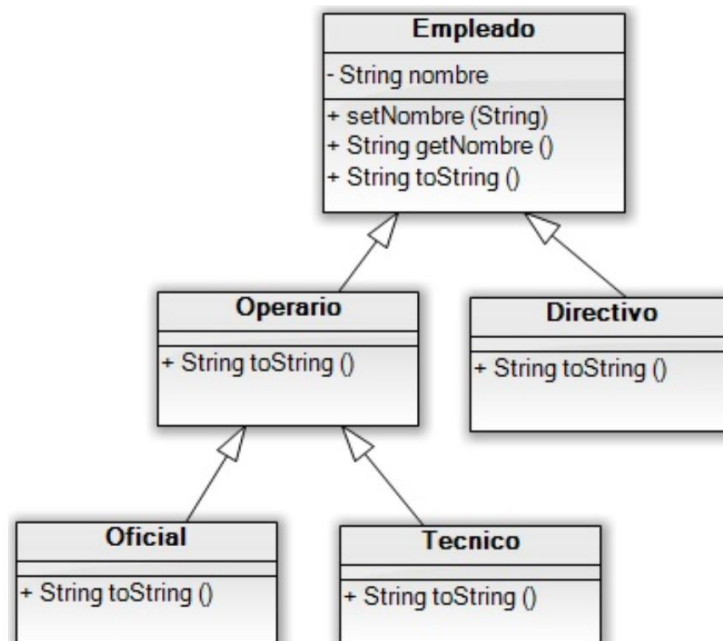
1. Mostrar número de entradas libres
2. Vender entradas
3. Salir

Ejercicio 2. (Sobrecarga de metodos)

Escriba una clase en la que se defina un método que escriba por pantalla el tipo de argumento que se le pasa como parámetro. Ejemplo: si se le pasa un carácter (char), que imprima algo del tipo "Soy un carácter". Esta clase deberá ser capaz de detectar todos los tipos primitivos de Java.

Ejercicio 3 (Herencia).

Realiza el siguiente ejercicio:



Clase Main

```
Empleado E1 = new Empleado("Rafa");
Directivo D1 = new Directivo("Mario");
Operario OP1 = new Operario("Alfonso");
Oficial OF1 = new Oficial("Luis");
Tecnico T1 = new Tecnico("Pablo");
System.out.println(E1);
System.out.println(D1);
System.out.println(OP1);
System.out.println(OF1);
System.out.println(T1);
```

Salida por consola:

```
Empleado Rafa
Empleado Mario -> Directivo
Empleado Alfonso -> Operario
Empleado Luis -> Operario -> Oficial
Empleado Pablo -> Operario -> Tecnico
```

Ejercicio 4 (Interfaces y Herencia).

Ahora debes crear la interface de la clase Empleado del ejercicio 3 y usarla en tu código.

Ejercicio 5 (Propiedad estática con herencia).

Crea la clase Vehiculo, así como las clases Bicicleta y Coche como subclases de la primera. Para la clase Vehiculo, crea los atributos de clase vehiculosCreados y kilometrosTotales, así como el atributo de instancia kilometrosRecorridos.

Crea también algún método específico para cada una de las subclases. Prueba las clases creadas mediante un programa con un menú como el que se muestra a continuación:

VEHÍCULOS

=====

1. Anda con la bicicleta
2. Haz el caballito con la bicicleta
3. Anda con el coche
4. Quema rueda con el coche
5. Ver kilometraje de la bicicleta
6. Ver kilometraje del coche
7. Ver kilometraje total
8. Salir

Elige una opción (1-8):

Ejercicio 6 (Herencia).

A partir de la siguiente ejecución:

1. Determinar superclase y subclases
2. Crear un constructor en la superclase y llamarlo desde las clases subclases.
3. Crear una clase Main que muestre por pantalla lo mismo que la ejecución

usando tantos métodos como puedas:

run:

Vivienda{precio=120000.0, numerodehabitaciones=3, nombrebarriada=Barriada del Carmen, calle=Calle falsa 123, jardin=false, patio=true}

Vivienda{precio=150000.0, numerodehabitaciones=3, nombrebarriada=Barriada del Carmen, calle=Calle falsa 178}

Vivienda{precio=0.0, numerodehabitaciones=4, nombrebarriada=El Paseo de Consolacion}, calle=calle Av. de Italia 1290Balcón: Tiene balcón}

¿Tiene balcón el piso situado en El Paseo de Consolacion, con 4 habitaciones?: Tiene balcón

El precio de la casa situada en la Barriada del Carmen es de: 120000.0€. Esta vivienda se encuentra en la calle: Calle falsa 123. Debido a que incorpora algunos extras, situado en la Calle falsa 123, tiene un precio final de: 135000.0€

La casa situada en la barriada Barriada del Carmen cambió de nombre, ahora se llama: Avenida de Itálica.

El número de viviendas es: 3

BUILD SUCCESSFUL (total time: 0 seconds)

Ejercicio 7 (Interfaces).

Dada la siguiente interface:

```
package Ejercicio1;
public interface MetodosDePilotoFormula1 {
    String getEscuderia();
    void setEscuderia(String escuderia);
    String getNombre();
    void setNombre(String nombre);
    public String getApellido();
    void setApellido(String apellido);
    String getNacionalidad();
    void setNacionalidad(String nacionalidad);
    int getTrofeosGanados();
    void setTrofeosGanados(int trofeosGanados);
    int getNumeroEnParrilla();
    void setNumeroEnParrilla(int numeroEnParrilla);
    int getEdad();
    void setEdad(int edad);
    int getTiempoEnF1();
    void setTiempoEnF1(int tiempoEnF1);
    int getTiempoEnEscuderia();
    void setTiempoEnEscuderia(int tiempoEnEscuderia);
    void celebracionTriunfo();
    void circuito(String nombCircuito);
    void adelanta(String piloto);
    void gana();
    void averia();
    int nuevoTrofeo();
    void nuevaEscuderia(String nuevaEscuderia);
    //¿Debo crear un constructor en una interfaz? Razona:
    //Ej1PilotoDeFormula1 (String nomb, String ape, String esc, String nac);
}
```

Conseguir la siguiente salida por pantalla usando tantos métodos de la Interface como sea posible:

Salida por pantalla:

Bienvenidos al circuito de Nurburgring... y comienza la carrera.

Atención: Hamilton lleva el coche a boxes por un pinchazo

Fernando adelanta a Vettel y se pone primero.

¡¡¡Alonso cruza la línea de parrilla y gana!!!!

Ahora unas palabras del piloto Español. Muchas gracias al equipo Renault por este triunfo.

Y con este, Alonso lleva 1 trofeos este año.

Hamilton ha cambiado a la escudería Mercedes

Bienvenidos al circuito de Brasil... y comienza la carrera.

Sebastian adelanta a Fernando y se pone primero.

Atención: Alonso lleva el coche a boxes por un pinchazo

Lewis adelanta a Vettel y se pone primero.

¡¡¡Hamilton cruza la línea de parrilla y gana!!!!

Ahora unas palabras del piloto Inglés. Muchas gracias al equipo Mercedes por este triunfo.

Hamilton lleva 2 trofeos este año.

Y con este, Alonso lleva 2 trofeos este año.

Alonso lleva 2 trofeos este año.

Sorprendentemente: Alonso lleva 14 trofeos este año. Por ello, gana su cuarto campeonato.

Ejercicio 8 (Herencia y tipo enum).

Nos piden hacer gestionar una serie de productos.

Los productos tienen los siguientes atributos:

- Nombre
- Precio

Tenemos dos tipos de productos:

- Perecedero: tiene un atributo llamado días a caducar
- No perecedero: tiene un atributo llamado tipo

Crea sus constructores, getters, setters y toString.

Tendremos una función llamada calcular, que según cada clase hará una cosa u otra, a esta función le pasaremos un número siendo la cantidad de productos:

- En Producto, simplemente sería multiplicar el precio por la cantidad de productos pasados.
- En Perecedero, aparte de lo que hace producto, el precio se reducirá según los días que quedan para que caduque:
 - Si le queda 1 día para caducar, se reducirá 4 veces el precio final.
 - Si le quedan 2 días para caducar, se reducirá 3 veces el precio final.
 - Si le quedan 3 días para caducar, se reducirá a la mitad de su precio final.
- En NoPerecedero, hace lo mismo que en producto.

Crea una clase ejecutable y crea un array de productos y muestra el precio total de vender 5 productos de cada uno. Crea tú mismo los elementos del array.

Ejercicio 9 (Herencia y arrays de objetos).

Crearemos una supeclase llamada Electrodomestico con las siguientes características:

Sus atributos son precio base, color, consumo energético (letras entre A y F) y peso. Indica que se podrán heredar.

Por defecto, el color será blanco, el consumo energético será F, el precioBase es de 100 € y el peso de 5 kg. Usa constantes para ello.

Los colores disponibles son blanco, negro, rojo, azul y gris.

Los constructores que se implementarán serán:

- Un constructor por defecto.
- Un constructor con el precio y peso. El resto por defecto.
- Un constructor con todos los atributos.

Los métodos que implementará serán:

Métodos get de todos los atributos.

- comprobarConsumoEnergetico(char letra): comprueba que la letra es correcta, sino es correcta usará la letra por defecto. Se invocará al crear el objeto y no será visible.

-comprobarColor(String color): comprueba que el color es correcto, sino lo es usa el color por defecto. Se invocará al crear el objeto y no será visible.

-precioFinal(): según el consumo energético, aumentara su precio, y según su tamaño, también.

Estas son las listas de precios:

LETRA	PRECIO
A	100 €
B	80 €
C	60 €
D	50 €
E	30 €
F	10 €

TAMAÑO	PRECIO
Entre 0 y 19 kg	10 €
Entre 20 y 49 kg	50 €
Entre 50 y 79 kg	80 €
Mayor que 80 kg	100 €

Crearemos una subclase llamada Lavadora con las siguientes características:

- Su atributo es carga, además de los atributos heredados.
- Por defecto, la carga es de 5 kg. Usa una constante para ello.
- Los constructores que se implementarán serán:
 - Un constructor por defecto.
 - Un constructor con el precio y peso. El resto por defecto.
 - Un constructor con la carga y el resto de atributos heredados. Recuerda que debes llamar al constructor de la clase padre.

Los métodos que se implementara serán:

- Método get de carga.
- precioFinal(): si tiene una carga mayor de 30 kg, aumentara el precio 50 €, sino es así no se incrementará el precio. Llama al método padre y añade el código necesario. Recuerda que las condiciones que hemos visto en la clase Electrodomestico también deben afectar al precio.

Crearemos una subclase llamada Television con las siguientes características:

Sus atributos son resolución (en pulgadas) y sintonizador TDT (booleano), además de los atributos heredados.

Por defecto, la resolución será de 20 pulgadas y el sintonizador será false.

Los constructores que se implementarán serán:

- Un constructor por defecto.
- Un constructor con el precio y peso. El resto por defecto.
- Un constructor con la resolución, sintonizador TDT y el resto de atributos heredados.

Recuerda que debes llamar al constructor de la clase padre.

Los métodos que se implementarán serán:

- Método get de resolución y sintonizador TDT.
- precioFinal(): si tiene una resolución mayor de 40 pulgadas, se incrementará el precio un 30% y si tiene un sintonizador TDT incorporado, aumentara 50 €. Recuerda que las condiciones que hemos visto en la clase Electrodomestico también deben afectar al precio.

Ahora crea una clase ejecutable que realice lo siguiente:

- Crea un array de Electrodomesticos de 10 posiciones.
- Asigna a cada posición un objeto de las clases anteriores con los valores que desees.
- Ahora, recorre este array y ejecuta el método precioFinal().
- Deberás mostrar el precio de cada clase, es decir, el precio de todas las televisiones por un lado, el de las lavadoras por otro y la suma de los Electrodomesticos (puedes crear objetos Electrodomestico, pero recuerda que Television y Lavadora también son electrodomésticos).

Por ejemplo, si tenemos un Electrodomestico con un precio final de 300, una lavadora de 200 y una televisión de 500, el resultado final será de 1000 (300+200+500) para electrodomésticos, 200 para lavadora y 500 para televisión.

Ejercicio 10. Repaso.

Nos piden realizar una agenda telefónica de contactos.

Un contacto está definido por un nombre y un teléfono (No es necesario de validar). Un contacto es igual a otro cuando sus nombres son iguales.

Una agenda de contactos está formada por un conjunto de contactos (Piensa en que tipo puede ser)

Se podrá crear de 2 formas, indicando nosotros el tamaño o con un tamaño por defecto (10)

Los métodos de la agenda serán los siguientes:

- anadirContacto(Contacto c): Añade un contacto a la agenda, sino se pueden meter más a la agenda se indicará por pantalla. No se pueden meter contactos que existan, es decir, no podemos duplicar nombres, aunque tengan distinto teléfono.

- existeContacto(Contacto c): indica si el contacto pasado existe o no.
- listarContactos(): Lista toda la agenda
- buscaContacto(String nombre): busca un contacto por su nombre y muestra su teléfono.

- `eliminaContacto(Contacto c)`: elimina el contacto de la agenda, indica si se ha eliminado o no por pantalla.

- `agendaLlena()`: indica si la agenda está llena.

- `huecosLibres()`: indica cuantos contactos más podemos meter.

Crea un menú con opciones por consola para probar todas estas funcionalidades.